

FULL-APP ANALYSIS · BRUTALLY HONEST EDITION · 2026-07-21

Peak Fettle: the whole system, graded without the pep talk.

Mobile app (89.8k LOC) + LifeOS companion (21.3k LOC) + API server (13.4k LOC), reviewed from the working tree at commit 5272a88 — architecture, product, code quality, testing, security, shipping readiness, and process. Every claim below was verified in the repo, not quoted from docs.

TL;DR — Remarkable product engineering. Not yet a shipped product, and not yet a business.

Prepared by Claude (Fable 5) from direct codebase inspection. No numbers in this deck are self-reported by past sessions.

What was actually reviewed

Surface area measured

Mobile app (mobile/)

89,834 LOC · 260 TS/TSX files · 41 screens + 8 tabs

LifeOS companion (lifeos/)

21,333 LOC · 106 files · 22 screens + 5 tabs

API server (peak-fettle-agents/)

13,356 LOC · 48 JS files · 20 route modules

Database

Supabase Postgres (drifted from db/schema.sql by design)

Infra

Railway (API, auto-deploy on push) · EAS build + OTA updates

Tests

28 mobile + 7 LifeOS + 6 server test files (all pure-logic)

How it was checked (this session)

Live prod probe:

API /health on Railway returned 200 — the 2026-07-14 outage is over.

Type health:

tsc --noEmit run fresh — mobile: 63 errors, LifeOS: 0 errors.

Monetization sweep:

grep for IAP / RevenueCat / Stripe / receipts across all three codebases.

Security spot-checks:

auth.js hashing, rate limiters, tier-mutation gating read line-by-line.

Invariant audit:

local-first leak grep (17 screens import api/*; spot-checked branching).

Docs as claims, not truth:

roadmaps/reviews used for context; every status re-verified in code.

The scorecard

A-	Architecture & data model	Local-first tier split, E2E-encrypted backup, on-device stats model. Genuinely rare design.
A	Product differentiation	Verified empty market at the tracker x screen-time-gate intersection; free tier beats every rival.
A-	Feature depth	Supersets, circuits, autoregulation, importers, watch, i18n (2,322 keys) - competitive with 10-yr incumbents.
B+	Security posture	bcrypt-12, scoped rate limits, secure-by-default tier endpoint. No 2FA; single HS256 JWT secret.
B-	Code quality	Strong modules undermined by 3,682-line god components, 63 tolerated tsc errors, dead files in main.
C	Testing	Good math coverage; zero component/E2E tests. The recurring bug classes are exactly the untested kind.
F	Observability	No crash reporting, no analytics, no feedback channel. Post-launch you are completely blind.
D	Shipping & monetization	Internal-only builds, no store presence, no payment rail at all. Pro is a free env-flag toggle.
C-	Process & bus factor	One human + agent swarm; real verification gates, but incidents keep proving the tree is fragile.

THE SYSTEM

Three codebases, one server, one invariant

Peak Fettle (mobile/)

89.8k LOC - Expo SDK 54 / RN 0.81.5, new arch
49 screens: logger, plans, rankings, groups, health, insights, importers, watch bridge
Free tier: on-device SQLite only (schema v15)
Pro tier: Postgres + PowerSync sync

LifeOS (lifeos/)

21.3k LOC - habits, goals, focus/blocking, mood, direction model (207-case test suite)
Shares auth + Pro entitlement with mobile
Blocker = Swift FamilyControls targets, never compiled (Windows dev machine)

API server (Node/Express)

13.4k LOC - 20 route modules on Railway
Supabase Postgres; drift-tolerant SQL
bcrypt-12, JWT HS256, scoped rate limits
Serves Pro sync, groups, share links, percentile legacy path, LifeOS pings

THE CENTRAL INVARIANT - AND THE #1 HISTORICAL BUG SOURCE

Free users make ZERO personal REST calls: everything lives in on-device SQLite, with E2E-encrypted backup (AES-256-GCM + human recovery code).
This invariant is the whole privacy brand - and it is enforced only by developer discipline plus a grep step in CLAUDE.md. It has leaked at least six times (infinite spinners, 500s, slow cold starts). It held in the screens spot-checked today, but nothing in CI structurally prevents leak #7.

Architecture decisions most teams never attempt

- **Local-first free tier is real, not marketing.** tierPolicy.ts branches every data path; free workout data physically cannot leak from a server that never receives it. JEFIT's 9M-account breach class is architecturally impossible here.
- **On-device strength percentiles (strengthModelV3).** Lognormal-fit, DOTS-calibrated, age-adjusted rankings computed offline. No competitor offers this at all - and it works with zero users on the platform.
- **A deterministic training engine that shows its work.** Seeded-PRNG reproducible plan generation with rule_trace explanations, while the whole category gets publicly roasted for fake 'AI'. Autoregulation + fatigue modeling landed with 58 tests.
- **Drift-tolerant server SQL.** Routes catch 42P01/42703 and degrade instead of 500ing against the known-drifted prod schema. Unglamorous and exactly right.
- **Resilience learned from real outages.** The 2026-07-14 Railway outage took the API down for days - and free users never noticed. Auth cold-start won't clear tokens on transient failures. OTA-first ship path (expo-updates, fingerprint runtime) is wired in both apps.



WHAT IS GENUINELY EXCELLENT

Product position (verified against the July market scan)

DIMENSION	THE MARKET	PEAK FETTLE
Free tier	Strong: 3 routines. Hevy: ads + 3-mo history. Fitbod: none.	Unlimited logging, plans, rankings, calculators, export - no ads
Privacy	Gymverse discloses health data as tracking; Hevy ships 4 ad SDKs.	Free data never reaches a server; E2E backup; no ad SDKs
Programming	'AI' coaches at \$35/mo that are rules engines in a trench coat.	Honest deterministic engine, free, explainable
Strength context	Nobody offers percentiles.	Offline percentile + tier ladder - unique in category
Screen-time gate	Step counters & camera pushups (ClearSpace, Steppin).	Only product gating on a real, loaded, progressing workout

The caveat that haunts this whole slide: every one of these advantages is currently invisible, because the app has never shipped publicly.

There is no business here yet - structurally

- **No payment rail exists. Anywhere.** Zero hits for IAP, StoreKit, RevenueCat, Stripe, or receipt validation across all three codebases. 'Pro' is flipped by POST /user/upgrade behind the ALLOW_CLIENT_TIER_TOGGLE env flag - a documented beta escape hatch. Nobody can give you money today even if they beg.
- **The app has never been distributed.** Every EAS profile is distribution: internal. No App Store submission, no external TestFlight, no Play listing. Google OAuth consent is still in Testing mode (test users only).
- **Zero real users, 125k lines of code.** The build-to-validation ratio is badly inverted: 19 tickets (watch app, Live Activities, Siri intents, localization scaffold) landed in a two-day sprint with no user feedback loop on the other end. That is inventory, not traction.
- **The competitive window is real and open - for now.** The July analysis found exactly one competitor attempting the tracker x gate combo ('Lock In'), still shallow. Every month of not shipping donates lead time to a funded copycat.

Plainly: this is currently a very sophisticated hobby. The gap between 'impressive repo' and 'product' is one TestFlight submission plus one RevenueCat integration - weeks of work that keep losing priority to feature construction.

You are flying blind, and local-first makes it worse

- **No crash reporting.** No Sentry, Bugsnag, or Crashlytics in either app. The iOS 26 TurboModule boot crash burned days precisely because there was no telemetry - it was debugged by founder-reproduction. The next native crash in the field will be invisible until a human complains.
- **No analytics of any kind.** No PostHog/Amplitude/anything. Retention, funnel drop-off, which of the ~49 screens anyone opens, whether autoregulation suggestions get accepted - all unknowable. Every roadmap prioritization so far has been vibes plus competitor grep.
- **Local-first cuts both ways.** For free users the server sees nothing - which is the brand - but it means you cannot even infer usage from API logs. Privacy-respecting, on-device-aggregated analytics (opt-in) is compatible with the brand promise; having literally nothing is not a virtue, it is a blindfold.
- **No feedback channel in-app.** Beta feedback historically arrived as pasted markdown files from testers. There is no structured path from a user's frustration to your backlog.

Grade F is not hyperbole: observability is the single cheapest fix in this deck (a day of work) with the highest information payoff, and it has been skipped for 14 months of development.

A mountain of code no device has ever executed

- **The device-test backlog is long and load-bearing.** Never run on hardware: Apple Watch pairing (Stage A), Live Activities (whose host module was silently a no-op facade for a while - caught only by re-audit), HealthKit under iOS 26 (needs the cold-start x20 test), progress-photo deletion, pseudo-locale truncation.
- **Swift cannot compile on the dev machine.** All LifeOS blocker targets and the watch app are Windows-authored Swift that has never seen xcodebuild. The first macOS compile will surface signature errors by the project's own admission (native/README.md).
- **'Code landed' has repeatedly meant 'not actually working'.** The project's history is a catalog of shipped-but-broken: push delivery failed silently twice (PUSH-001/002), OAuth builds crashed at login pre-rebuild, the Live Activity could never start. The parse-sweep gate catches syntax, not behavior.
- **Everything queues behind one scarce resource.** Native verification waits on 'the EAS batch' - build minutes and founder device time. Risk compounds: when that batch finally runs, failures from seven tickets will land simultaneously and be hard to attribute.

Hotspots: the code is better than average and worse than it thinks

God components (lines, single file)



The logger - the product's heart - is one 3,682-line file. Every fix costs full-file comprehension.

Accepted rot

- 63 tsc errors tolerated as 'baseline' (fresh count today; policy is only 'don't add more')
- templates_copy_test.tsx - a dead copy-paste screen committed to main
- Working tree dirty right now: staged deletions + modified auth files sitting uncommitted - the exact foot-gun that already caused one prod outage

FRAGILE INVARIANTS HELD BY CONVENTION, NOT TOOLING

- Local-first: enforced by a grep instruction in CLAUDE.md. Has regressed repeatedly; nothing fails CI when a screen calls the API on the free path.
- Units: weight must round-trip through units.ts; a missed conversion once stored 185 lb as 185 kg. Still nothing but convention prevents recurrence.
- Column-name traps documented instead of fixed: fcm_token stores an Expo token; server weight_kg is an alias that 500s if queried directly (caused a P0).
- Schema truth is split: prod DB has drifted from db/schema.sql, and correctness depends on every future migration remembering to be drift-tolerant.

The tests cover the math. The bugs live in the app.

- **What is tested is tested well.** Training engine v1/v2, autoregulation, fatigue, streaks, importers, migrations, direction model (207 cases), badge evaluator - pure-logic suites, ~248 green at last full gate. This is real and better than most indie apps.
- **What is not tested is where every P0 has occurred.** Zero component, integration, or E2E tests across 49 screens and 90k LOC of UI. The historical bug classes - local-first REST leaks, unit conversion, safe-area double-insets, duplicate-workout races, OAuth login crashes - are all integration-level. The test suite is structurally incapable of catching the next one.
- **Server coverage is thin.** 6 test files for 20 route modules / 13.4k LOC. Auth, groups, sets - the money paths - are verified by curl-and-hope plus incident response.
- **The verification gate is syntax, not behavior.** The parse-sweep DoD exists because agents kept committing files that would not parse. It is necessary here and catches real corruption - but 'all files parse' is the floor of software quality, and it is currently doing duty as the ceiling.

Process: one human, an agent swarm, and a messy repo

- **Bus factor: exactly one.** ~125k LOC written substantially by AI agents, reviewed by a single founder. The CLAUDE.md 'lessons' file is genuinely excellent institutional memory - and also a 400-line confession of how many times the process has hurt itself: corruption incidents, truncated commits shipped to main, stale-index commits wiping a day of work (restored in 420e4ff), a staged-deletion sweep that took prod down.
- **The repo root is a junk drawer.** ~120 files at top level: 30+ dated roadmap/prompt files, the founder's personal resume (plus LaTeX .aux/.log artifacts), logo experiments, and at least five dead directories (peak-fettle-app, MyApp, mind, qml, orchestrator, ollama-agents). This is friction for every future session and a real leak risk if the repo is ever shared.
- **Push-to-main is production deploy.** No staging environment, no PR review, no branch protection. Every push to main deploys the API to Railway immediately. Combined with the dirty-index history, the safety margin on the server is one typo wide.
- **The discipline that exists is reactive.** Every safeguard (parse-sweep, drift-tolerance, commit skill) was built after the corresponding disaster. Credit for building them - but the pattern predicts the next control will also be purchased with an outage.

LifeOS: best code in the repo, hostage to Apple

- **Code quality is the strongest in the monorepo.** 0 TypeScript errors (vs mobile's 63). The direction model shipped with a 207-case determinism suite and a graded-citation derivation doc. Safety scaffolding (crisis banner, disclaimer gate, no-runtime-AI, per-day-boolean privacy) is thoughtful and consistent.
- **Its reason to exist is a feature that has never run.** The app-blocking core is Swift FamilyControls code that has never been compiled, on an entitlement Apple has not granted. If Apple says no - which happens routinely for FamilyControls distribution requests - LifeOS is a habit tracker in a market full of free habit trackers.
- **It is Pro-anchored, and Pro cannot be bought.** LifeOS access is the paid tier's anchor value (the right call strategically) - but with no payment rail and no store listing, that anchor is currently attached to nothing.
- **Notifications half-exist.** Server-side LO_* types are registered; queue writers and the reminder-time UI were deferred. A habit app whose reminders do not fire is a demo.
- **Verdict:** hold. Do not invest further LifeOS feature work until (a) the entitlement request is actually submitted and answered, and (b) one macOS compile proves the Swift targets build. Both are founder actions, not agent actions, and both have been pending since June.

Ranked by expected damage

CRITICAL	Ship-delay spiral: building features instead of shipping	Open - 19 tickets landed since the last real user feedback
CRITICAL	FamilyControls entitlement denied -> LifeOS thesis collapses	Open - request not yet submitted (founder action since June)
HIGH	Field crash with zero telemetry -> silent churn	Open - no Sentry in either app; iOS 26 crash was the warning shot
HIGH	Local-first leak #7 ships to the free tier	Mitigated by convention only; needs a lint/CI rule
HIGH	Deferred device-test batch fails seven tickets at once	Open - watch/LA/HealthKit/photos all unverified on hardware
MEDIUM	Solo bus factor + push-to-main = prod outage	Recurring - has already happened; no branch protection
MEDIUM	Prod schema drift bites a future migration	Partially mitigated - drift-tolerant patterns, canonical schema.sql
MEDIUM	Copycat with funding ships the tracker x gate combo first	External - window verified open as of July 2026

The next six moves, in order - and what NOT to do

- **1. Ship to external TestFlight this month.** Everything else in this deck is theory until strangers hold the app. The EAS batch is already the plan; make it the ONLY priority. Store assets, privacy labels, review notes - boring, unavoidable, overdue.
- **2. Add Sentry + opt-in analytics before those testers arrive.** One day of work. Without it, the TestFlight cohort teaches you almost nothing.
- **3. Integrate RevenueCat and turn off the free-Pro env flag.** The /user/upgrade endpoint was explicitly designed to be called by a payment webhook - the socket is wired, plug it in. Even at 0 sales, a real paywall makes the tier system honest.
- **4. Submit the FamilyControls entitlement request today.** It costs nothing, the answer gates the entire LifeOS bet, and it has been 'day-1 action' for five weeks.
- **5. Turn the two recurring bug classes into lint rules.** An ESLint rule banning api/* imports outside the data layer, and one banning raw weight math outside units.ts. An afternoon each; ends the two most-repeated P0 families forever.
- **6. Freeze new features until the above lands.** Not because the features are bad - the watch app and autoregulation are good work - but because the marginal feature is worth ~0 users while the marginal shipping task is worth all of them.

BOTTOM LINE

The hard part is done. The scary part hasn't started.

As engineering, this is a top-decile solo project: a defensible architecture (local-first privacy, on-device statistics, honest deterministic programming), feature depth that stands next to decade-old incumbents, and a verified gap in the market that no funded competitor has filled.

As a product, it does not yet exist: no store listing, no way to pay, no telemetry, and a growing pile of hardware-dependent code that has never touched hardware. The failure mode from here is not technical - the codebase will bear a launch. The failure mode is another six months of exquisite, unverified inventory while 'Lock In' or someone like them takes the category story.

Ship the TestFlight. Wire the paywall. Send the entitlement request. Then - and only then - build the next feature.

Overall grade: A- product engineering / D go-to-market. The average of those two numbers is chosen by what you do next month.